UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Facultat d'Informàtica de Barcelona

FIB

FAST PARALLEL TOKEN INFERENCE WITH LANGUAGE DIFFUSION MODELS

ALBERT GOMEZ TRIUNFANTE

Thesis supervisor
PABLO AGUSTIN MARTIN TORRES (Department of Computer Science)

Tutor: JAVIER BÉJAR ALONSO (Department of Computer Science)

Degree
Bachelor's Degree in Informatics Engineering (Information Technologies)

Bachelor's thesis

Facultat d'Informàtica de Barcelona (FIB)

Universitat Politècnica de Catalunya (UPC) - BarcelonaTech

Contents

1 Introduction and contextualization

The field of Natural Language Processing (NLP) is currently dominated by Auto-Regressive (AR) Transformers (e.g., GPT-5, DeepSeek-V3, Kimi), which generate text sequentially. While effective, this paradigm suffers from a linear latency bottleneck: generating N tokens requires N forward passes, leading to slow inference and suboptimal GPU utilization.

Language Diffusion Models (LDMs) [1] have emerged as a parallel alternative, theoretically capable of generating entire sequences simultaneously via iterative denoising. However, current LDMs still require a high number of sampling steps (typically 50–100) to reach convergence, which often negates their speed advantage over AR models.

In the domain of Computer Vision (Image Generation), this limitation was overcome by techniques such as **Progressive Distillation** [2] or Consistency Models [3], where a "Student" model is trained to compress the "Teacher's" many denoising steps into a single or very few steps.

Problem Statement: While distillation techniques have revolutionized image generation, they have not yet been successfully adapted to the discrete and non-continuous domain of text diffusion.

Thesis Proposal & Institutional Context: Within the specialization of **Information Technology (IT)** at the Facultat d'Informatica de Barcelona (FIB), this project aims to implement and validate a **Knowledge Distillation** technique for Language Diffusion Models. By training a student LDM to mimic the trajectory of a pre-trained teacher LDM, we aim to achieve high-quality text generation in extremely few steps (< 10), thereby unlocking the true potential of parallel token inference. Because deploying large-scale language models currently faces severe practical and economic constraints, this project is directly anchored in core IT themes: by drastically reducing the required denoising steps, we aim to maximize inference efficiency and significantly increase system throughput in production environments.

1.1 The Auto-Regressive vs. Diffusion Paradigm

Standard AR models estimate $P(x) = \prod P(x_t|x_{<t})$. This serial dependency prevents parallelization. LDMs, conversely, model the data distribution $p(x_0)$ by reversing a noise process $q(x_t|x_{t-1})$.

$$x_{t-1} \leftarrow \text{Denoise}(x_t, \theta) \tag{1}$$

While LDMs can predict all tokens in parallel, the iterative refinement requires K steps. If K is large, the latency advantage vanishes.

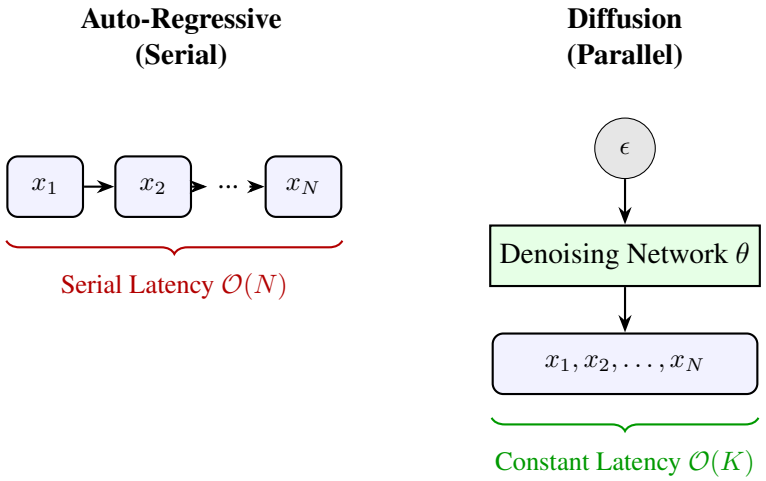

Figure 1: Latency Comparison. **Left:** AR models generate tokens sequentially. **Right:** Diffusion models generate the entire sequence in parallel. *Source: Compiled by the author.*

1.2 Current Limitations and Challenges in Discrete Diffusion

While models like GPT-4 have set the standard for quality, their serial nature poses a fundamental limit on latency scaling. The time complexity of generating a sequence of length N is $\mathcal{O}(N)$. As context windows grow to 128k or 1M tokens, this serial dependency becomes a bottleneck that no amount of parallel compute can solve purely through hardware scaling. Techniques like *Speculative Decoding* attempt to mitigate this by drafting multiple tokens at once, but they are probabilistic and their speedup is upper-bounded (typically $2\times - 3\times$). Diffusion models, by contrast, offer a theoretical $\mathcal{O}(1)$ or $\mathcal{O}(K)$ complexity where K is the number of denoising steps, independent of sequence length N.

As of early 2026, research on LDMs (e.g., LLaDA, MDLM) focuses on architecture and pre-training [4]. **There is a significant gap in post-training acceleration**[5]. No open-source study has comprehensively applied progressive distillation to discrete text tokens, a fact that reflects how relatively new and less understood text diffusion is compared to its image counterpart. This introduces significant challenges:

- **Rounding Errors:** In text, a slight error in the embedding space can flip a token from "happy" to "sad," destroying semantic coherence.
- **Loss Mismatch:** The standard Mean Squared Error (MSE) loss used in image distillation does not align well with the Cross-Entropy loss required for language modeling.

1.3 Stakeholders

Table 1: Stakeholder Analysis Matrix

Stakeholder	Interest (Motivation)	Power (Influence)
Project Director	High (Academic success, publication)	High (Approves scope/grade)
The Student	High (Learning, Career portfolio)	High (Executor)
Scientific Community	High (Reproducibility of new method)	Low (Passive observer)
Industry (SMEs)	Medium (Cost reduction in API)	Low (Potential future adopters)

Detailed Analysis:

- **Project Director:** Their primary concern is the feasibility of the scope. By strictly defining the boundaries (no pre-training, only distillation), we mitigate the risk of scope creep, ensuring the project is "finishable" within the TFG timeline.
- **The Student:** This project serves as a portfolio piece demonstrating mastery of advanced Deep Learning concepts (Diffusion, Distillation, PyTorch) that are highly sought after in top-tier AI research labs and tech companies.
- **Scientific Community:** If successful, the open-source code release will allow other researchers to replicate our distillation pipeline, potentially sparking a new wave of "Fast Text Diffusion" research.

Strategy: We will manage the *Director* closely via bi-weekly meetings to align on the "Distillation Loss" formulation, while keeping the *Scientific Community* in mind by ensuring the code is modular and open-source compatible.

2 Background

This section provides the theoretical foundations required to understand the proposed distillation technique. We begin with continuous diffusion models as originally formulated for image generation, then derive how the framework is adapted to discrete token sequences. We then review the Transformer architecture that serves as the backbone for all models considered, and conclude with an overview of knowledge distillation and the specific parameter-efficient fine-tuning technique used in this work.

2.1 Continuous Diffusion Models

Diffusion probabilistic models [6] define a **forward process** that gradually corrupts a data sample $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ by adding Gaussian noise over T discrete time steps:

$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}\Big(\mathbf{x}_t;\ \sqrt{1-\beta_t}\,\mathbf{x}_{t-1},\ \beta_t\mathbf{I}\Big)\,, \tag{2}$$

where $\{\beta_t\}_{t=1}^{T}$ is a fixed variance schedule. A key property of this Markov chain is that any intermediate noisy sample can be sampled in closed form:

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}\big(\mathbf{x}_t;\ \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0,\ (1-\bar{\alpha}_t)\mathbf{I}\big)\,, \quad \bar{\alpha}_t = \prod_{s=1}^{t}(1-\beta_s). \tag{3}$$

The **reverse process** learns to undo this corruption step by step. A neural network $\epsilon_\theta(\mathbf{x}_t, t)$ is trained to predict the noise ϵ added at step t, minimizing the denoising score-matching objective:

$$\mathcal{L}_{\text{DDPM}} = \mathbb{E}_{t,\mathbf{x}_0,\epsilon}\Big[\big\|\epsilon - \epsilon_\theta\big(\sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon,\ t\big)\big\|^2\Big]\,, \quad \epsilon \sim \mathcal{N}(\mathbf{0},\mathbf{I}). \tag{4}$$

This loss is a re-weighted Evidence Lower BOund (ELBO) on the log-likelihood $\log p_\theta(\mathbf{x}_0)$. At inference time, generation proceeds by iterating the learned reverse step from pure noise $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0},\mathbf{I})$ down to $\mathbf{x}_0$, requiring T (typically 1000) sequential network evaluations.

DDIM and faster samplers. Song et al. [7] showed that the reverse process can be reformulated as a non-Markovian process sharing the same marginals, enabling deterministic sampling in as few as 50 steps with negligible quality loss. Despite this improvement, the number of steps still scales independently of sequence length, and each step is a full forward pass of a large network.

2.2 Discrete Diffusion Models for Text

Text consists of discrete tokens drawn from a finite vocabulary $\mathcal{V}$ of size V (typically $V \approx 32{,}000$ for modern tokenizers). Applying Gaussian noise to integer token indices is semantically meaningless; the continuous diffusion framework must therefore be adapted to operate over categorical distributions.

2.2.1 Masked Diffusion

The dominant paradigm for discrete text diffusion replaces Gaussian corruption with a **masking** process [1], [8]. Given a sequence $\mathbf{x}_0 = (x_0^1, \ldots, x_0^L)$, the forward process independently masks each token with probability $\alpha(t)$ using a special `[MASK]` token:

$$q(x_t^i \mid x_0^i) = \begin{cases} \texttt{[MASK]} & \text{with probability } 1-\alpha(t), \\ x_0^i & \text{with probability } \alpha(t), \end{cases} \tag{5}$$

where $\alpha(t)$ is a monotonically decreasing schedule with $\alpha(0) = 1$ (no masking) and $\alpha(T) \approx 0$ (fully masked). The reverse model $p_\theta(x_0^i \mid \mathbf{x}_t)$ is trained to predict the original token at each masked position given the full (partially masked) context, minimizing the cross-entropy loss:

$$\mathcal{L}_{\text{mask}} = \mathbb{E}_{t,\mathbf{x}_0}\left[\sum_{i:\, x_t^i = \texttt{[MASK]}} -\log p_\theta\big(x_0^i \mid \mathbf{x}_t\big)\right]. \tag{6}$$

Because all masked positions are predicted simultaneously in a single forward pass, this formulation achieves the parallel generation property that motivates diffusion models for text.

2.2.2 LLaDA

Large Language Diffusion with mAsking (LLaDA) [1] is the primary teacher model in this work. It is an 8B-parameter Transformer pre-trained from scratch on 2.3T tokens using the masked diffusion objective of Eq. (6). At inference, LLaDA uses a **re-masking** sampling strategy: starting from a fully masked sequence, it iteratively predicts all tokens and then re-masks a fraction of low-confidence predictions, refining the sequence over T steps. This is analogous to the reverse diffusion chain in the continuous case. The key challenge — and the central motivation for this thesis — is that LLaDA requires $T = 64$ to 128 steps to achieve competitive output quality, each step involving a full 8B-parameter forward pass.

2.2.3 MDLM

Masked Diffusion Language Models (MDLM) [8] derives a continuous-time extension of the masked diffusion process, showing that the ELBO can be computed in closed form and that the model can be trained with a simple weighted cross-entropy loss equivalent to Eq. (6). MDLM also demonstrates that the masking schedule can be parameterized as a linear function $\alpha(t) = 1 - t/T$, $t \in [0, T]$, which we adopt in this work.

2.3 The Transformer Architecture

All language models considered in this thesis are built on the Transformer architecture [9]. A Transformer processes a sequence of L token embeddings $\mathbf{H} \in \mathbb{R}^{L\times d}$ through N stacked layers, each composed of two sub-modules:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V, \tag{7}$$

where $Q = \mathbf{H}W^Q$, $K = \mathbf{H}W^K$, $V = \mathbf{H}W^V$ are linear projections of the hidden states, and d_k is the key dimension. Unlike AR Transformers, which apply a causal mask to prevent tokens from attending to future positions, diffusion Transformers use **bidirectional** (full) attention: every token attends to every other token. This is essential for parallel token prediction, as the model must gather global context before committing to a prediction at each position.

Positional encoding. Modern large-scale Transformers replace sinusoidal positional encodings with Rotary Position Embeddings (RoPE) [10], which encode relative position information directly in the attention computation and generalize better to sequence lengths unseen during training.

Time conditioning. In the diffusion setting, the denoising model must be aware of the current noise level t. This is achieved by embedding the scalar t using sinusoidal features and adding the resulting vector to the hidden states at each layer, analogously to how DDPM conditions the UNet on t [6]. LLaDA instead conditions on the masking ratio $1 - \alpha(t)$ directly.

2.4 Knowledge Distillation

Knowledge distillation (KD) [11] is a model compression technique in which a compact **student** model f_S is trained to replicate the behavior of a larger, pre-trained **teacher** model f_T. Rather than training the student on hard one-hot labels, it is trained on the **soft probability distributions** output by the teacher, which encode richer inter-class similarity information. For a classification head with logits z, the soft target distribution at temperature τ is:

$$p_T^i = \frac{\exp(z_T^i/\tau)}{\sum_j \exp(z_T^j/\tau)}, \tag{8}$$

and the student is trained to minimize the KL divergence from teacher to student:

$$\mathcal{L}_{\text{KD}} = \tau^2 \cdot D_{\text{KL}}(p_T \,\|\, p_S) = \tau^2 \sum_i p_T^i \log \frac{p_T^i}{p_S^i}. \tag{9}$$

The factor τ^2 compensates for the gradient magnitude reduction introduced by the temperature scaling. In this work, the teacher and student share the same architecture (same parameter count); the goal of distillation is not model compression but **step compression**: we train the student to perform in one step what the teacher performs in two, progressively halving the required number of inference steps.

2.4.1 Progressive Distillation for Diffusion Models

Salimans and Ho [2] introduced progressive distillation (PD) as a technique to reduce the number of sampling steps in continuous diffusion models by a factor of two at each distillation round. The procedure is as follows:

1. Initialize the student $\theta_S \leftarrow \theta_T$ (copy teacher weights).
2. For each training batch, sample a noisy state $\mathbf{x}_t$ and run *two* teacher denoising steps to obtain a target $\hat{\mathbf{x}}_0$.
3. Train the student to reach $\hat{\mathbf{x}}_0$ in a *single* step from $\mathbf{x}_t$, using the MSE loss in the data prediction parameterization.
4. Set $\theta_T \leftarrow \theta_S$ and halve the step schedule. Repeat until the desired number of steps is reached.

After k rounds, a model originally requiring N steps is compressed to $N/2^k$ steps. This procedure is well-defined in continuous space because the interpolation between two denoising steps is smooth. Adapting it to discrete token sequences — where there is no meaningful interpolation between categorical distributions — is the central technical challenge addressed by this thesis.

2.5 Parameter-Efficient Fine-Tuning: LoRA

Full fine-tuning of an 8B-parameter model on a single A100 GPU is infeasible: the model weights alone require ≈16 GB in `bfloat16`, leaving minimal headroom for activations and optimizer states. Low-Rank Adaptation (LoRA) [12] addresses this by decomposing the weight update ΔW of each linear layer into a product of two low-rank matrices:

$$W' = W + \Delta W = W + BA, \quad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k},\ r \ll \min(d,k). \tag{10}$$

The original weights W are frozen; only A and B are trained. For a rank $r = 16$ applied to all attention projections of an 8B model, the number of trainable parameters drops from ≈8B to ≈20M — a reduction of over 99%. A is initialized with random Gaussian entries and B with zeros, so $\Delta W = 0$ at the start of training, preserving the teacher's behavior at initialization. This is a critical property for our distillation setup, as it ensures the student begins training from the teacher's exact output distribution.

3 Methodology

This section describes the concrete implementation of the progressive distillation pipeline used in this thesis. The exposition documents the implemented pipeline components: teacher trajectory caching, student training with parameter-efficient adapters, and the evaluation suite.

3.1 Pipeline overview

The pipeline comprises two stages. First, the teacher (LLaDA) runs in a read-only generation mode to produce and cache intermediate denoising states and the corresponding teacher logits. Each cached file contains an input tensor for the student, the teacher's vocabulary logits at the chosen intermediate step, and meta-information such as the prompt length. In the experimental configuration adopted in this work the teacher uses $T = 128$ total denoising steps and the cache targets the midpoint $t = 64$, with a generated continuation length of $L_{\text{gen}} = 64$.

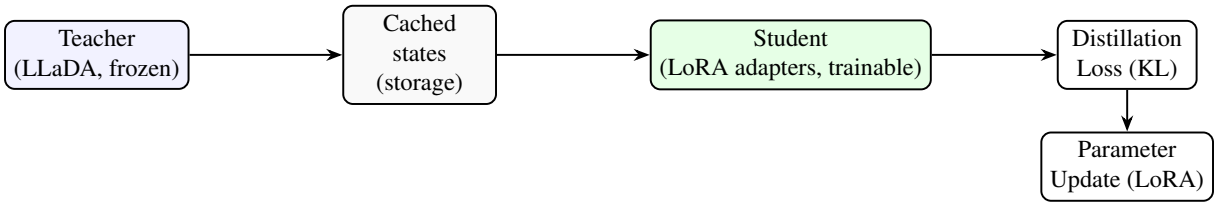

Figure 2: Two-stage distillation pipeline for teacher caching and LoRA-based student distillation.

The second stage initializes the student from the same pre-trained weights, injects low-rank adapters into attention projections, and optimizes only these adapters so that the bulk of the 8B model parameters remain frozen. Cached trajectories are consumed sequentially (one cached trajectory per update), i.e. an effective batch size of $B = 1$ is used. The student is trained to match the teacher's soft-token distributions over the generated continuation (positions following the prompt).

3.2 Distillation loss and gradient flow

Following classical KD, the student minimizes the temperature-scaled KL divergence per token between the teacher and the student distributions (compare Eq. (9) in Background). For a batch of sequences the loss used is:

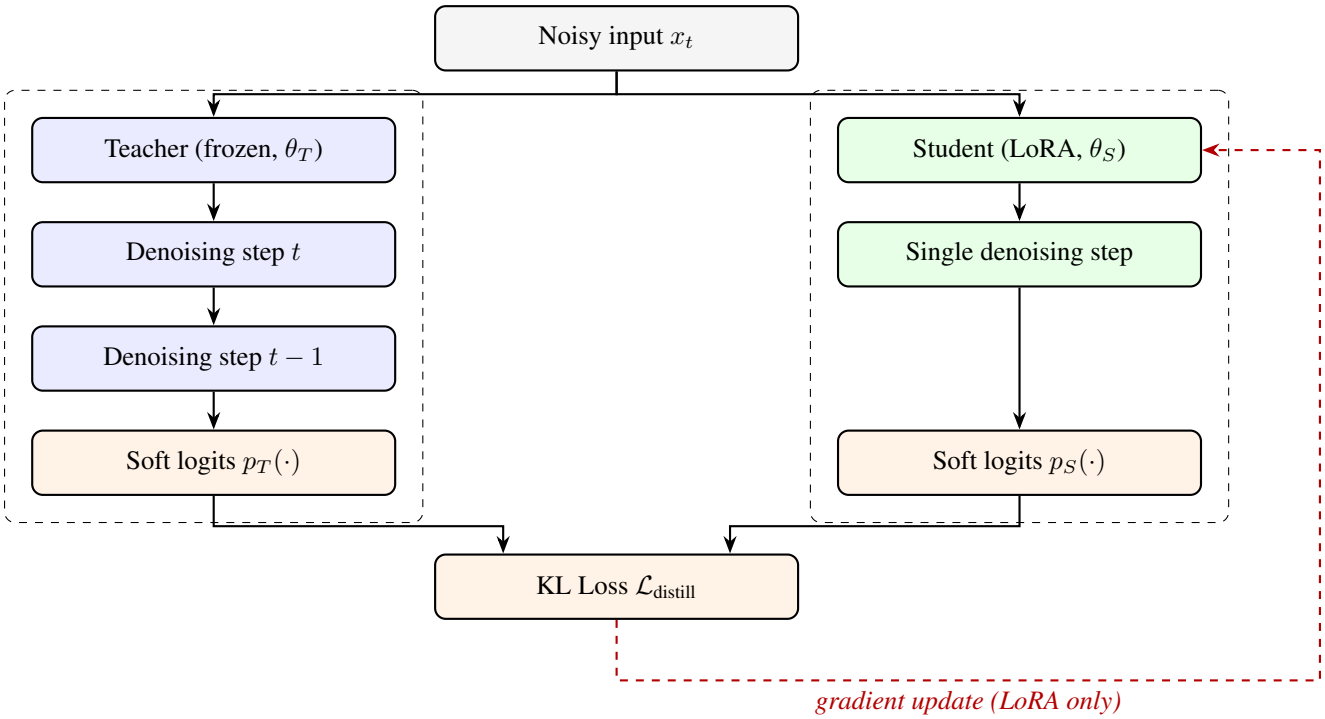

Figure 3: Knowledge Distillation Architecture

$$\mathcal{L}_{\text{distill}} = \tau^2 \cdot \frac{1}{B} \sum_{b=1}^{B} \sum_{t=1}^{L_b} D_{\text{KL}}\big(p_T(\cdot \mid b,t) \parallel p_S(\cdot \mid b,t)\big), \tag{11}$$

where p_T and p_S are the teacher and student soft-token distributions at a generation position, τ is the temperature (a temperature of $\tau = 2.0$ was selected for the experiments reported here) and B is the batch size (in the implemented runs $B = 1$). The loss is computed only over generation positions following the prompt so the student learns to predict the generative tail while keeping prompt-conditioning intact.

Because text tokens are categorical, the student is trained directly on the soft teacher logits stored in the cached trajectories rather than on hard re-sampled token indices. This avoids the need to backpropagate through discrete sampling operations and reduces variance, stabilizing distillation for categorical outputs.

3.3 Step compression strategy

The central objective of progressive distillation is to compress the inference-time step budget of a pre-trained teacher model by a factor of two at each distillation round, progressively building a chain of increasingly fast student models without retraining from scratch. This section describes the mechanics of this process in detail, covering the step-halving schedule, the trajectory construction that makes each round feasible in the discrete setting, the multi-round chaining procedure, and a theoretical analysis of the approximation error accumulated across rounds.

3.3.1 The Progressive Halving Schedule

We follow the progressive halving schedule of Salimans and Ho [2]: each distillation round attempts to compress two teacher steps into a single student step. If the teacher initially requires N steps, after k rounds the student operates with

$$M = \frac{N}{2^k}. \tag{12}$$

The schedule is defined over a discrete, uniformly spaced time grid $\{0, \Delta, 2\Delta, \ldots, N\Delta\}$ where $\Delta = T/N$ is the step size and T is the total noise level at the fully masked state. Round k uses a grid of $N_k = N/2^k$ steps with step size $\Delta_k = 2^k\Delta$. The student trained in round k is therefore asked to jump twice as far in noise-level space as the student from round $k-1$, which is why each successive round requires a fresh distillation phase rather than merely rescaling an existing sampler.

Why halving, not quartering? A halving factor is the minimal non-trivial compression ratio that remains stable in practice. Each round introduces an approximation error bounded by the curvature of the denoising trajectory over the merged interval; halving keeps this interval small enough that the teacher's two-step output is still a reliable supervision target for the student. Larger compression factors (e.g., $4\times$ per round) can be applied, but empirically lead to larger quality degradation, because the student must cover a region of the trajectory where the denoising dynamics are no longer well approximated by a single network evaluation.

3.3.2 Trajectory Construction for Discrete Tokens

In the continuous setting, the teacher's two-step output is a point in $\mathbb{R}^{L\times d}$ that serves directly as the regression target for the student's MSE loss. In the discrete masked-diffusion setting, the analogue is the teacher's **soft token distribution** after two unmasking steps, which lives in the simplex Δ^{V-1} at each of the L_{gen} generated positions.

Concretely, let $\mathbf{x}_t$ be a partially masked sequence at noise level t (i.e., a fraction $1-\alpha(t)$ of positions carry the `[MASK]` token). One teacher step produces:

$$\hat{\mathbf{x}}_{t-1} = \text{sample}(p_{\theta_T}(x_0 \mid \mathbf{x}_t)), \tag{13}$$

where the teacher predicts the *original* token at every masked position and then re-masks a fraction $1 - \alpha(t-1)$ of those predictions to produce $\hat{\mathbf{x}}_{t-1}$. A second teacher step from $\hat{\mathbf{x}}_{t-1}$ yields the logits $\ell_T \in \mathbb{R}^{L_{\text{gen}} \times V}$ used as the distillation target:

$$\ell_T = f_{\theta_T}(\hat{\mathbf{x}}_{t-1}), \quad p_T(\cdot \mid b, i) = \text{softmax}\Big(\ell_T^{b,i}/\tau\Big). \tag{14}$$

Because the re-masking step in Eq. (13) is stochastic, the trajectory is *cached*: for each training example the teacher is run once offline and both $\mathbf{x}_t$ (the student input) and p_T (the soft target) are written to disk. This ensures that the student sees a consistent supervision signal throughout its training run, avoiding instability from on-the-fly teacher sampling.

Handling fully unmasked positions. Positions that are not masked in $\mathbf{x}_t$ already contain the correct token. The distillation loss is computed *only* over the L_{gen} generated positions (i.e., those following the prompt), matching the convention in Eq. (11). Prompt positions are excluded from the loss to prevent the student from being penalized for reproducing already-known tokens and to keep the gradient signal focused on the generative tail.

3.3.3 Multi-Round Chaining and Teacher Promotion

A key feature of progressive distillation is that the procedure is *iterative*: after the student of round k has converged, it is *promoted* to become the teacher of round $k+1$. Formally, let $\theta_T^{(k)}$ denote the teacher weights at round k. The promotion rule is:

$$\theta_T^{(k+1)} \leftarrow \theta_S^{(k)}, \tag{15}$$

where $\theta_S^{(k)}$ is the student weight snapshot at convergence of round k. The LoRA adapters trained in round k are *merged* into the base weights before promotion, so that round $k+1$ begins with a standard dense model rather than a base-plus-adapter stack. This is achieved via the standard LoRA weight-merge formula:

$$W^{(k+1)} = W_{\text{base}} + B^{(k)} A^{(k)}, \tag{16}$$

where $A^{(k)}, B^{(k)}$ are the low-rank adapter matrices trained in round k. Fresh LoRA adapters (with B re-initialized to zero) are then injected into the merged model before the next round, restoring the zero-initialization invariant described in Section 2.5.

Table 2 summarizes the planned round progression for this thesis, starting from the LLaDA teacher at $N = 128$ steps and targeting $N = 8$.

Table 2: Planned progressive distillation rounds.

Round k	**Teacher steps** N_k	**Student steps** M_k	**Compression**	**Notes**
0 (baseline)	128	–	–	LLaDA pre-trained
1	128	64	$2\times$	First distillation
2	64	32	$4\times$	Teacher = Round-1 student
3	32	16	$8\times$	
4	16	8	$16\times$	Target configuration

3.3.4 Convergence Criterion and Early Stopping

Each distillation round is run until one of two conditions is met. The primary criterion is a plateau in the token-level KL divergence between teacher and student:

$$\overline{\text{KL}}_k = \frac{1}{|\mathcal{D}_{\text{eval}}|} \sum_{(\mathbf{x}_t,\, p_T) \in \mathcal{D}_{\text{eval}}} \frac{1}{L_{\text{gen}}} \sum_{i=1}^{L_{\text{gen}}} D_{\text{KL}}(p_T(\cdot \mid i) \parallel p_S(\cdot \mid i)). \tag{17}$$

Training is stopped when $\overline{\mathrm{KL}}_k$ has not decreased by more than $\epsilon = 0.01$ nats over a window of 200 consecutive optimizer steps. The secondary criterion is a hard upper bound on the total number of optimizer steps per round (5,000 steps in the experimental configuration), which prevents runaway training in cases where the student fails to converge due to a poorly calibrated learning rate.

If the primary convergence criterion is met but $\overline{\mathrm{KL}}_k$ remains above a quality threshold $\delta_{\mathrm{KL}} = 0.5$ nats, the round is flagged as a partial success and the resulting student is not promoted. In that case, the learning rate is reduced by a factor of 0.5 and training is resumed for one additional window of 2,000 steps before a final promotion decision is made.

3.3.5 Error Accumulation Across Rounds

Each distillation round introduces an approximation error ϵ_k between the student's distribution and the teacher's two-step target. Over K rounds this error accumulates, and understanding its growth is important for assessing the viability of aggressive compression targets (e.g., $128 \to 8$ steps, requiring $K = 4$ rounds).

Let $\epsilon_k = \overline{\mathrm{KL}}_k$ (the per-round KL at convergence). Under the assumption that successive errors are independent (which holds approximately when the LoRA adapters are re-initialized between rounds), the total distributional divergence between the original teacher and the round-K student can be bounded by the triangle inequality for KL divergence in the following loose sense:

$$D_{\mathrm{KL}}\left(p_T^{(0)} \parallel p_S^{(K)}\right) \;\leq\; \sum_{k=1}^{K} \epsilon_k. \tag{18}$$

In practice, errors do not accumulate linearly because later rounds start from a student that already approximates the original teacher well, and the denoising dynamics at low step counts are smoother (fewer masked positions remain, so predictions are more confident). Empirically in the image diffusion literature, the quality loss per round is sub-linear, meaning that a four-round compression from 128 to 8 steps incurs substantially less than $4 \times \epsilon_1$ total degradation.

Mitigation via temperature calibration. To reduce the risk of distributional drift across rounds, the distillation temperature τ is adjusted at each round. Specifically, τ is increased slightly (by a factor of 1.1 per round) to produce softer teacher targets, which are easier for the student to match and result in a smaller ϵ_k at the cost of marginally lower sharpness in the student's output distribution. This schedule is a deliberate conservative choice: sharpness can be recovered at inference time by applying a low temperature to the student's logits, whereas large per-round errors cannot be corrected after training.

3.4 LoRA integration and hyperparameters

To fit an 8B model in limited GPU memory we apply Low-Rank Adaptation as in Eq. (10) and inject adapters only into attention projections. The chosen LoRA configuration for the experiments reported here is:

- $r = 8$
- $\alpha_{\text{LoRA}} = 16$
- target modules: `["q_proj","v_proj"]`
- dropout = 0.05

Adapter parameters are cast to `float16` prior to training to reduce VRAM and optimizer-state requirements.

Only LoRA parameters are trainable; the base model weights are frozen and the teacher is loaded in 4-bit quantized mode using `BitsAndBytesConfig` to allow inference on a single A100-class GPU. This combination enables a practical trade-off between fidelity to the teacher and resource constraints.

3.5 Training protocol and implementation details

The student training loop uses an AdamW optimizer; when an 8-bit optimizer implementation is available the pipeline switches to the lower-precision variant to reduce optimizer memory. The learning rate used is 1×10^{-4}, global gradient norms are clipped to 1.0, and mixed precision (`float16`) is used for adapter parameters. Gradient checkpointing is enabled when supported by the model to reduce activation memory.

Trajectories are generated from the Wikitext-2-raw-v1 dataset: a small subset of longer examples is sampled, each prompt is expanded by $L_{\text{gen}} = 64$ tokens, and the resulting trajectory tensors are cached to disk. Each cached trajectory is saved as a single file and consumed sequentially by the trainer, with one optimizer step performed per file, yielding an effective batch size of $B = 1$. This design simplifies memory management and increases robustness to per-sample failures.

3.6 Evaluation protocol

The evaluation suite combines three complementary measurement axes: language modelling quality, task-level correctness, and inference efficiency. All routines are orchestrated by a single evaluation script that coordinates the sub-evaluators and aggregates results into a leaderboard across distillation rounds.
Quality metrics.

- **Perplexity (PPL):** computed using a GPT-2 reference model over a held-out prompt set. GPT-2 is used as a fixed, widely adopted reference scorer rather than as a baseline system; lower perplexity indicates that the student's generations are more consistent with natural language.
- **KL-divergence:** the mean KL divergence between the teacher's and student's output token distributions at each generated position, averaged over the evaluation prompts. This measures distillation fidelity directly, independently of surface-form quality.

Task-level correctness and LLM-as-judge. Task-level evaluation is carried out via Promptfoo [13], an assertion-based evaluation framework. A configuration file specifies a set of prompted test cases together with Python-based assertion functions; each assertion calls a locally hosted judge model (Llama 3.1 8B, served via Ollama) that returns a binary Yes/No verdict on properties such as coherence, factual consistency, and instruction following. Promptfoo aggregates assertion pass rates across all test cases and produces a structured JSON report, from which an overall task-correctness score is derived. The teacher model is evaluated under the same protocol and serves as the primary quality baseline.
Efficiency metrics.

- **Wall-clock inference latency:** measured in tokens per second on a fixed hardware configuration across a standardised prompt set.
- **Profiling traces:** caching and training scripts export Chrome-compatible profiler traces via `torch.profiler`, enabling fine-grained per-operator analysis of the training and inference pipelines.

Baseline. The unmodified teacher model (LLaDA at $T = 128$ denoising steps) is evaluated under the same quality and task-level protocol and constitutes the primary reference point for all distillation rounds.

4 Justification of the chosen resolution alternative

4.1 Methodological Choice: Progressive Distillation vs. Alternatives

When seeking to accelerate diffusion models, two primary strategies emerge: Consistency Models [3] and Progressive Distillation [2]. While Consistency Models map any point on the diffusion trajectory directly to the origin in a single step, they are notoriously difficult to adapt to discrete categorical domains like text. They rely on enforcing boundary conditions over continuous Ordinary Differential Equations (ODEs).

In contrast, **Progressive Distillation** teaches a student model to take larger, discrete jump steps (e.g., matching 2 teacher steps in 1 student step). This discrete nature makes it significantly more adaptable to the token-by-token embedding space of language models. Therefore, this technique was selected as the most viable path to bridge the gap shown in Table 3.

Table 3: Comparison of Generation Paradigms and the Thesis Gap

Paradigm	Example Model	Generation	Steps (N)	Latency	Quality
Auto-Regressive	GPT-4, LLaMA-3	Serial	$N = \text{Length}$	High (Linear)	SOTA
Standard Diffusion	SSD-LM, LLaDA	Parallel	$50 - 100$	Med-High	High
Continuous Distillation	Stable Diffusion XL *(image only)*	Parallel	$4 - 8$	Low	High
Discrete Distillation	**(This Thesis)**	**Parallel**	**4 - 8**	**Very Low**	**Target: High**

4.2 Scientific Innovation

Unlike a standard engineering project that benchmarks existing tools, this thesis proposes a **novel adaptation of an algorithm**. If successful, it contributes a new method to the NLP community: "Textual Progressive Distillation."

4.3 Efficiency and Green AI

Reducing the number of inference steps linearly reduces the energy required to generate text. If we reduce steps from 64 to 8, we reduce the energy cost by approximately $8\times$. This aligns with the "Green AI" initiative [14] to make Large Language Models sustainable.

4.4 Commercial Viability

For real-time applications (e.g., conversational AI), latency is the primary KPI. A parallel model running in 4-8 steps could theoretically outperform LLaMA-3 by $5\times$–$10\times$ in wall-clock speed, making it highly valuable for industry. Consider a customer support chatbot that needs to generate a 200-token response:

- **AR Model (LLaMA-3):** At 50 tokens/sec, this takes 4 seconds. This latency is noticeable and degrades user experience.
- **Standard LDM:** At 100 steps, it might also take 4 seconds or more.
- **Distilled LDM (Our Proposal):** With 4 steps, even if each step is heavier, the total latency could drop to 0.4 seconds.

This order-of-magnitude improvement ($10\times$) transforms the user experience from "waiting" to "instant," enabling new applications like real-time voice-to-voice translation where latency must be sub-500ms.

4.5 Economic Impact

Current LLM inference is prohibitively expensive. Companies like OpenAI and Anthropic spend millions daily on compute. A reduction in inference steps from 50 to 5 represents a potential $10\times$ reduction in GPU compute time per query. For an SME deploying a custom LDM, this technique could be the difference between a viable product and a bankrupt one.

5 Scope of the project

5.1 General and Specific Objectives (SMART)

The general objective is to implement and evaluate a **knowledge distillation technique** for Language Diffusion Models, training a "Student" model to compress the inference steps of a "Teacher" model, thereby achieving fast parallel text generation.

- **O1 (Theoretical Framework):** Analyze the mathematical formulation of Progressive Distillation (Images) and adapt the loss function for discrete text data (Categorical Cross-Entropy / KL Divergence) by **March 15, 2026**.
- **O2 (Implementation):** Develop a "Teacher-Student" training loop in PyTorch, capable of loading a pre-trained LDM (e.g., MDLM/LLaDA) and fine-tuning the student weights, by **April 1, 2026**.
- **O3 (Distillation Training):** Train the student model on a subset of the Wikitext-2-raw-v1 dataset to reduce sampling steps from $N = 64$ to $N = \{8, 4\}$, utilizing university or cloud GPU resources.
- **O4 (Validation):** Benchmark the "Distilled LDM" against the "Teacher LDM" and an "AR Baseline" (LLaMA-3), measuring **Speedup (x)** vs. **Quality Drop (Perplexity/BLEU)** by **May 20, 2026**.

5.2 In-Scope vs. Out-of-Scope

In-Scope:

- **Algorithm Adaptation:** Porting the distillation logic (step halving) from the continuous domain to the discrete text domain.
- **Model Distillation (Fine-tuning):** We assume the existence of a pre-trained "Teacher". We will not pre-train a model from scratch; we will only perform the distillation (fine-tuning) phase.
- **Target Models:** Open-source LDMs in the 1B-8B parameter range (e.g., MDLM, SSD-LM, LLaDA).
- **Evaluation:** Comparing the distilled model against the teacher on standard metrics (PPL, BLEU) and efficiency metrics (Latency, FLOPs).

Out-of-Scope:

- **Pre-training from Scratch:** We will not train the base LDM. We rely on published weights.
- **Novel Architectures:** We are not inventing a new Neural Network architecture (e.g., new Attention mechanism), but rather a new *training method* for existing architectures.
- **Production Deployment:** The output is a scientific proof-of-concept, not a production API.

5.3 Functional and Non-Functional Requirements

5.3.1 Functional Requirements

- The system must accept a text prompt as input and generate a coherent text continuation.
- The distillation pipeline must be able to load a pre-trained Teacher model and initialize a Student model with identical weights.
- The training loop must implement a specific loss function that handles discrete categorical data (e.g., Cross-Entropy with Soft-Targeting).

5.3.2 Non-Functional Requirements

- **Performance:** The distilled model must achieve a latency reduction of at least $4\times$ compared to the teacher.
- **Quality:** The Perplexity (PPL) degradation of the student model must not exceed 15% relative to the teacher.
- **Resources:** The fine-tuning process must fit within a single NVIDIA A100 (40GB/80GB) GPU using LoRA optimization.

5.4 Validation Strategy and Metrics

To ensure the scientific validity of our results, we will employ a rigorous evaluation protocol:

1. **Teacher-Student Gap:** We will measure the KL-divergence between the teacher's output distribution and the student's. A low divergence indicates successful distillation.
2. **Human Evaluation (Proxy):** We will use a strong LLM (e.g., GPT-4-Turbo) as a judge to rate the coherence of the distilled text compared to the teacher's text, identifying potential "hallucinations" introduced by the step reduction.
3. **A/B Testing:** We will run side-by-side comparisons of latency on different batch sizes ($B = 1, B = 32$) to determine if the speedup holds under high-load production scenarios.

6 Time planning

6.1 Global Project Parameters and Methodology

This project is formulated as a standard Bachelor's Thesis (TFG) for the Facultat d'Informatica de Barcelona (FIB), corresponding to 18 ECTS credits. Therefore, the total workload is strictly planned for **540 hours**. It spans from February 10, 2026, to the expected defense on June 25, 2026 ($\sim$ 136 days), requiring approx. 27.5 hours/week.

This project adopts an **Agile Experimental Methodology**, adapted for research. Unlike a linear Waterfall approach, research requires iterative refinement of hypotheses. We divide the project into 2-week "Sprints." At the end of each sprint, a Sprint Review will be held with the Project Director to assess the achieved milestones, followed by a Sprint Retrospective to adjust the planning for the subsequent sprint based on empirical results.

6.2 Work Breakdown Structure (WBS)

To ensure granular tracking and mitigate risks, no single task in this planning exceeds the **50-hour maximum threshold** mandated by the GEP guidelines.

Concurrent Tasks (Cross-cutting) These tasks span the entire duration of the project.

- **PM1: GEP Course Deliverables [40h].** Involves drafting, formatting, and refining the Context, Planning, Budget, and Final GEP deliverables. The 40-hour estimate is justified as each of the 4 major documents requires approximately 10 hours of dedicated writing and formatting.
- **PM2: Weekly Sprint Planning & Director Meetings [20h].** Involves preparing agendas and reviewing progress. The 20-hour estimate corresponds to one hour per week over the $\sim$ 20-week duration of the project.
- **DOC1: State of the Art drafting [25h].** Reading selected papers and synthesizing them into the academic framework. 25 hours is necessary due to the dense mathematical nature of recent diffusion models.
- **DOC2: Methodology & Experiments drafting [35h].** Documenting the code architecture, loss functions, and experimental setups. This takes 35 hours because it runs continuously alongside Sprints 2, 3, and 4.
- **DOC3: Final memory review and formatting [20h].** Proofreading, LaTeX compilation fixes, and bibliography checks before final submission.

Sprint Breakdown

- **Sprint 1: Theoretical Setup (Feb 10 - Mar 8) [80h]**

  - **T1.1: Literature review on Discrete Diffusion [30h].** This task involves reading roughly 15 key papers to extract current methodologies. It requires 30 hours because adapting continuous image algorithms to discrete text requires a deep understanding of complex probability flows.
  - **T1.2: Mathematical formulation of Loss Function [30h].** Deriving the exact equations to be coded later (e.g., KL-Divergence soft-targets). The 30-hour estimate is justified by the heavy mathematical adaptation required to bridge the gap between continuous MSE and categorical Cross-Entropy.
  - **T1.3: Teacher model validation [20h].** Downloading the LLaDA weights and setting up a basic inference script to verify the open-source model works. 20 hours is assigned to handle potential undocumented environment dependencies.

- **Sprint 2: Pipeline Implementation (Mar 9 - Apr 5) [90h]**
  - **T2.1: Setup PyTorch environment [20h].** Configuring CUDA, HuggingFace libraries, and resolving dependency conflicts on the local PC.
  - **T2.2: Implement DistillationTrainer class [35h].** Writing the core Python class that will handle the forward passes of both Teacher and Student models. This demands 35 hours because it requires overriding standard PyTorch training loops to implement custom backward passes and gradient approximations.
  - **T2.3: WandB integration & Toy Task [35h].** Integrating logging and verifying convergence on a tiny dataset (TinyShakespeare). This takes 35 hours as it acts as the primary debugging phase for the mathematical logic coded in T2.2.

- **Sprint 3: Scale-up & Initial Training (Apr 6 - May 3) [90h]**
  - **T3.1: LoRA configuration for 8B model [30h].** Setting up Low-Rank Adaptation matrices to avoid VRAM overflow. 30 hours is justified as finding the optimal rank (r) and target modules requires several trial-and-error runs on the cloud GPU.
  - **T3.2: Initial training run (N=16) & gradient debugging [40h].** The actual distillation process. This is the most demanding task (40h) because discrete distillation is notoriously unstable; it requires active monitoring, pausing the run, and deploying immediate code patches to handle exploding gradients.
  - **T3.3: Hyperparameter tuning [20h].** Adjusting the learning rate and soft-target weights based on T3.2 results to ensure smooth loss convergence.

- **Sprint 4: Advanced Training & Benchmarks (May 4 - May 31) [90h]**
  - **T4.1: Final distillation runs (N=8, N=4 steps) [40h].** Executing the final training iterations to compress the model further. This requires 40 hours of passive monitoring and managing various model checkpoints over long cloud compute sessions.
  - **T4.2: Inference latency profiling [20h].** Using CodeCarbon and precise timers to measure tokens/second and energy consumption compared to the Teacher model.
  - **T4.3: Quality evaluation [30h].** Computing Perplexity (PPL) and BLEU scores on the Wikitext-2-raw-v1 dataset.
    30 hours is needed because calculating valid PPL on large datasets requires generating millions of tokens, which is highly time-consuming even on accelerated hardware.

- **Sprint 5: Wrap-up & Defense (Jun 1 - Jun 25) [50h]**
  - **T5.1: Final Results analysis and charting [20h].** Aggregating raw CSV data from T4.3 into professional academic plots using Matplotlib.

- **T5.2: Presentation slide deck creation [20h].** Condensing the 50-page thesis into a concise 15-minute visual presentation.
- **T5.3: Oral defense rehearsal [10h].** Practicing pacing and anticipating Q&A with the Project Director.

6.3 Task Summary and Resources Table

Table 4: Detailed Task Summary mapped to Required Resources

Code	Descriptive Name	Hours	Dependencies	Specific Resources
PM1	GEP Deliverables	40	None	Local PC, LaTeX, GEP Materials
PM2	Sprint Meetings	20	None	Local PC, Google Meet, Director
DOC1	State of the Art Draft	25	T1.1	Local PC, Mendeley, IEEE Xplore
DOC2	Methodology Draft	35	T3.2	Local PC, LaTeX
DOC3	Final Review	20	T4.3, T5.1	Local PC, Grammarly
T1.1	Literature Review	30	None	Local PC, arXiv
T1.2	Loss Formulation	30	T1.1	Local PC, Overleaf
T1.3	Teacher Validation	20	None	Local PC, HuggingFace
T2.1	Setup Environment	20	None	Local PC, Python, PyTorch
T2.2	DistillationTrainer	35	T1.2, T2.1	Local PC, GitHub
T2.3	Toy Task	35	T2.2	Local PC, WandB
T3.1	LoRA Configuration	30	T2.2	A100 GPU (Cloud/Cluster)
T3.2	Initial Training Run	40	T2.3, T3.1	A100 GPU, WandB
T3.3	Hyperparameter Tuning	20	T3.2	A100 GPU
T4.1	Final Distillation Runs	40	T3.3	A100 GPU, HuggingFace
T4.2	Latency Profiling	20	T4.1	Local PC / T4 GPU, CodeCarbon
T4.3	Quality Evaluation	30	T4.1	Local PC, BLEU/PPL Scripts
T5.1	Results Analysis	20	T4.2, T4.3	Local PC, Matplotlib
T5.2	Slide Deck Creation	20	T5.1	Local PC, PowerPoint/Beamer
T5.3	Defense Rehearsal	10	T5.2	Local PC, Project Director

6.4 Agile Gantt Chart

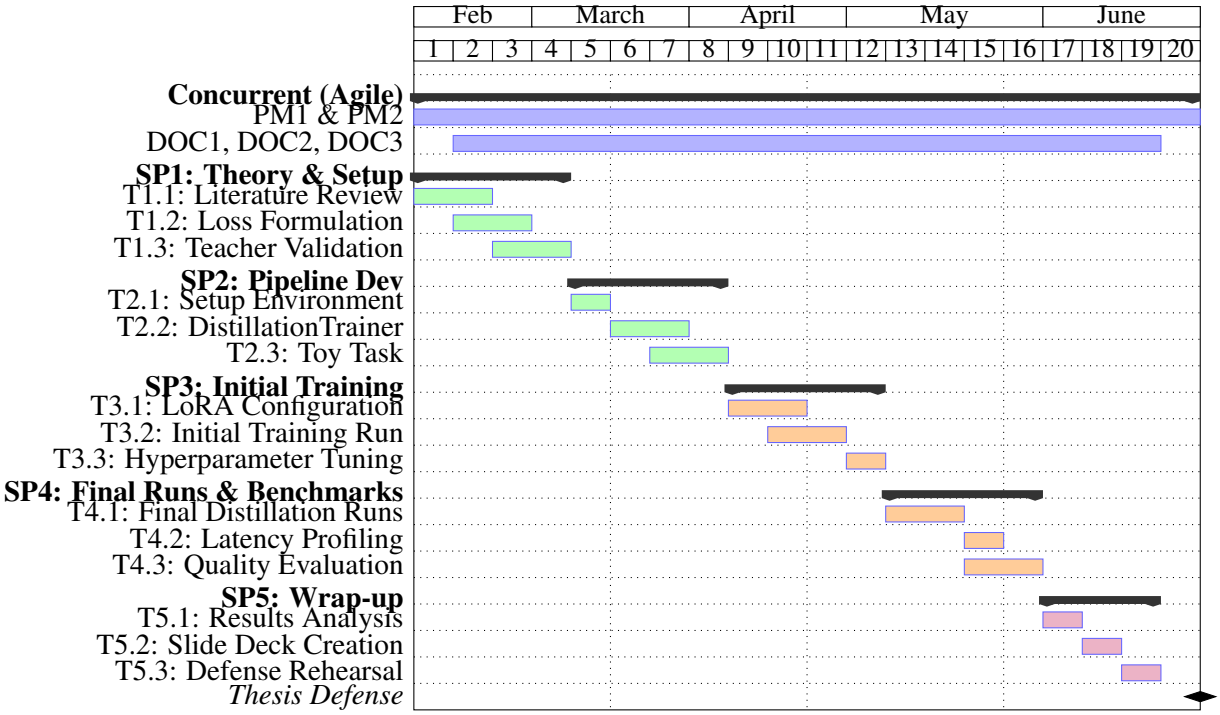

Figure 4: Granular Agile Gantt Chart mapping sub-tasks 1:1 to the WBS. *Source: Compiled by the author.*

6.5 Risk Management & Alternative Plans

As required by project management standards, we have defined "Plan B" tasks for critical risks. These alternative tasks replace primary tasks if a risk materializes, ensuring the project **still finishes by June 25**.

Risk 1: Severe VRAM Limitations (Hardware Failure). If the NVIDIA A100 GPU becomes unavailable or if the 8B parameter model Cannot fit in memory even with LoRA (during Sprint 3).

- **Plan B Task (ALT-T3.1):** Switch to a smaller Teacher model (e.g., a 1.5B or 3B diffusion model).
- **Time Impact & Schedule Adjustment:** Requires an estimated 15 hours to download, integrate, and validate the new model. We will absorb these 15 hours by reducing the scope of **T4.3** (Quality Evaluation). Instead of running extensive benchmarks on multiple datasets, we will restrict evaluation to a single dataset (Wikitext-2-raw-v1). The final delivery date remains unchanged.
- **Additional Resources Required:** Access to the HuggingFace Model Hub to locate and download the alternative architecture. It will also require consulting the specific API documentation for the new model, though no additional human resources (staff) are needed.

Risk 2: Distillation Training Instability. If the Student model's loss diverges completely or suffers from mode collapse during **T3.2** (Sprint 3).

- **Plan B Task (ALT-T3.2):** Abandon the KL-Divergence soft-target approach and revert to standard feature-matching loss (MSE on hidden states), which is vastly more stable but yields slightly worse text generation quality.
- **Time Impact & Schedule Adjustment:** Requires 20 hours to rewrite the loss function in the `DistillationTrainer` and restart the run. To recover these 20 hours, **T4.1** (Final Distillation

Runs) will be simplified. We will only distill to N=8 steps instead of testing both N=8 and N=4. The project successfully concludes on time with a slightly narrower scope.

- **Additional Resources Required:** This pivot will require an ad-hoc technical consultation (approx. 1-2 hours) with the Thesis Director to validate the new MSE approach. Additionally, because the failed run will have consumed initial budget, it requires re-allocating extra Cloud GPU (A100) time to execute the new training loop.

7 Budget

This section details the economic planning required to execute the thesis, adhering to the 540-hour workload defined in the project scope. Cost estimations include a standard 1.35 multiplier for Social Security contributions, referencing average gross salaries for Junior IT roles in Spain (Data: Glassdoor & InfoJobs, 2025).

7.1 Identification of Costs and Staff Estimates

To guarantee full granularity, the 540 hours have been broken down by every individual sub-task defined in the Gantt chart. Each task is mapped to the exact hourly cost of the assigned professional role.

Table 5: Highly Granular Staff Costs Broken Down by Individual Gantt Chart Task

Task Code	Assigned Role	Hours	Gross Rate	Rate w/ SS (x1.35)	Total Cost (€)
PM1	Project Manager	40	25.00 €/h	33.75 €/h	1,350.00
PM2	Project Manager	20	25.00 €/h	33.75 €/h	675.00
DOC1	AI Researcher	25	20.00 €/h	27.00 €/h	675.00
DOC2	AI Researcher	35	20.00 €/h	27.00 €/h	945.00
DOC3	AI Researcher	20	20.00 €/h	27.00 €/h	540.00
T1.1	AI Researcher	30	20.00 €/h	27.00 €/h	810.00
T1.2	AI Researcher	30	20.00 €/h	27.00 €/h	810.00
T1.3	ML Engineer	20	15.00 €/h	20.25 €/h	405.00
T2.1	ML Engineer	20	15.00 €/h	20.25 €/h	405.00
T2.2	ML Engineer	35	15.00 €/h	20.25 €/h	708.75
T2.3	ML Engineer	35	15.00 €/h	20.25 €/h	708.75
T3.1	ML Engineer	30	15.00 €/h	20.25 €/h	607.50
T3.2	ML Engineer	40	15.00 €/h	20.25 €/h	810.00
T3.3	ML Engineer	20	15.00 €/h	20.25 €/h	405.00
T4.1	ML Engineer	40	15.00 €/h	20.25 €/h	810.00
T4.2	ML Engineer	20	15.00 €/h	20.25 €/h	405.00
T4.3	AI Researcher	30	20.00 €/h	27.00 €/h	810.00
T5.1	AI Researcher	20	20.00 €/h	27.00 €/h	540.00
T5.2	Project Manager	20	25.00 €/h	33.75 €/h	675.00
T5.3	Project Manager	10	25.00 €/h	33.75 €/h	337.50
Total Staff Cost		**540**			**13,432.50**

7.2 General Costs and Amortization

General costs include hardware, software, and operational expenses. Hardware amortization is calculated over a standard 4-year (1,460 days) useful life, prorated for the 4.5 months ($\approx$ 136 days) of the project duration.

- **Hardware (Local PC):** Valued at 1,500 €. Amortization: $1,500 \times (136/1460) = 139.72$ €.
- **Hardware (Cloud GPU):** Rental of an NVIDIA A100 GPU for 100 hours of training at an estimated 2.50 €/h. This is a direct consumption cost: 250.00 €.
- **Software Licenses:** PyTorch, HuggingFace, and standard IDEs are open-source. GitHub Pro and Overleaf are covered via University Student Packs (0.00 €).
- **Operational Costs:** Electricity (540h $\times$ 0.5 kW $\times$ 0.20 €/kWh = 54.00 €) and workspace/internet overhead estimated at 150.00 €.

Total General Costs: 593.72 €

7.3 Contingencies and Incidentals

To ensure financial robustness, indirect costs are calculated based on standard IT risk margins and the specific risk matrix defined previously.

- **Contingency Margin:** A standard margin of **15%** of the cumulative direct costs (Staff + General, totaling 14,026.22 €) is applied to cover unforeseen general project delays. This yields a contingency reserve of **2,103.93 €**.
- **Incidentals:** Calculated based on specific risks, weighted by their probability:
  - *Risk 1 (VRAM limitations):* 15 extra hours of ML Engineer time (303.75 €). Estimated probability: 30%. Expected cost = 91.12 €.
  - *Risk 2 (Training Instability):* 20 extra hours of ML Engineer time (405.00 €). Estimated probability: 40%. Expected cost = 162.00 €.

7.4 Total Budget Summary

Table 6: Total Project Budget Summary

Concept	Amount (€)
Direct Costs	**14,026.22**
Human Resources	13,432.50
General Costs	593.72
Indirect Costs	**2,357.05**
Contingency Margin (15%)	2,103.93
Incidentals (Risk-weighted)	253.12
Total Project Budget	**16,383.27**

7.5 Management Control

To effectively control potential budget deviations during execution, financial indicators will be monitored at the end of every Agile Sprint. If the Cost Performance Index ($CPI = EV/AC$) drops below 0.90 at any sprint review, non-critical validation tasks (e.g., extensive multi-dataset benchmarking) will be descoped to re-align the budget dynamically without requiring additional funding.

8 Sustainability and social commitment

8.1 Self-Assessment Summary

Prior to completing the sustainability questionnaire, my understanding of sustainability in software engineering was largely limited to superficial hardware power consumption. The self-assessment revealed a significant blind spot in my perspective: I had not fully considered the socioeconomic implications of democratizing Artificial Intelligence, nor the full lifecycle costs of training Large Language Models (LLMs).

Strengths: A major strength identified during this assessment is my project's inherent alignment with "Green AI" principles; by focusing on Knowledge Distillation, the core objective is inherently sustainable (reducing inference compute).

Weaknesses: However, a weakness identified is the lack of a formal carbon-tracking protocol during my own model's training phase. Moving forward, I am committed to integrating tools like `CodeCarbon` into my pipeline to measure the actual CO_2 emissions of the distillation process itself, ensuring that the environmental cost of the "Student" training does not outweigh the benefits gained during inference. This reflection has transformed my view of the TFG from a purely technical algorithm-optimization task into a holistic Information Technology solution designed to solve real-world energy and accessibility bottlenecks.

8.2 Economic Dimension

- **Reflection on Estimated Cost:** The estimated project cost of $\approx$ 16,383 € is highly competitive when viewed as an R&D investment. Developing a custom accelerated Language Diffusion Model (LDM) from scratch would cost exponentially more in human and compute resources.
- **State of the Art (Current Solutions):** Currently, deploying LLMs in production is prohibitively expensive. Inference for autoregressive models scales linearly with token length, requiring massive GPU clusters running 24/7, costing companies thousands of euros daily.
- **Improvement:** By reducing inference steps from > 50 down to < 10, this distillation technique drastically cuts the server time required per query. This reduces the economic barrier to entry, allowing SMEs to deploy sophisticated language models without needing premium, high-tier GPU infrastructure.

8.3 Environmental Dimension

- **Project Impact & Minimization:** Training the distilled model will consume significant GPU power (estimated 100 hours on an A100). To minimize this impact, the project avoids training from scratch, instead reusing pre-trained weights via LoRA, which reduces trainable parameters by 99% and slashes training energy.
- **State of the Art (Current Solutions):** Standard AR models and non-distilled LDMs are environmentally taxing due to the massive energy draw (and subsequent cooling requirements) of data centers executing long, sequential generation passes.
- **Improvement:** This solution directly addresses the carbon footprint of AI inference. By requiring 80% fewer computational steps to generate the same text, the energy consumed per query drops proportionally, directly contributing to a lower global carbon footprint for AI services.

8.4 Social Dimension

- **Personal Growth:** This project forces me to master state-of-the-art Deep Learning concepts (Diffusion, PyTorch, Distillation) and bridge the gap between theoretical math and practical IT deployment, significantly enhancing my professional engineering profile.

- **State of the Art & Real Need:** Currently, the power of cutting-edge AI is concentrated in the hands of a few tech giants who can afford the hardware. There is a real societal need to democratize this technology to prevent technological monopolies.
- **Improvement:** By optimizing models to run faster and on less expensive hardware, this project contributes to making advanced NLP tools accessible to independent researchers, educators, and underfunded institutions, thereby improving equity in the tech landscape.

References

[1] S. Nie, D. Zhang, and X. Liu, "Large language diffusion models (llada): A new paradigm for parallel generation," *arXiv preprint arXiv:2502.09992*, 2025.

[2] T. Salimans and J. Ho, "Progressive distillation for fast sampling of diffusion models," *ICLR*, 2022.

[3] Y. Song and P. Dhariwal, "Consistency models," *ICML*, 2023.

[4] D. Zhang and Y. Li, "A comprehensive survey on diffusion language models: 2024-2026," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2026.

[5] X. Li, Y. Wang, et al., "Accelerating diffusion language models: A survey on sampling and post-training methods," *arXiv preprint arXiv:2406.10998*, 2024.

[6] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *Advances in Neural Information Processing Systems*, vol. 33, pp. 6840–6851, 2020.

[7] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," *ICLR*, 2021.

[8] S. Sahoo et al., "Masked diffusion language models," in *NeurIPS*, 2024.

[9] A. Vaswani et al., "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.

[10] J. Su et al., "Roformer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, 2024.

[11] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[12] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," *ICLR*, 2022.

[13] Promptfoo, *Promptfoo: Open-source llm testing and evaluation*, `https://www.promptfoo.dev`, 2024.

[14] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in nlp," University of Massachusetts Amherst, Tech. Rep., 2019.